CS6230 Project: QR Code Decoder

Released: Oct 15th 2026 • Due: *Nov 9, 2026*, 23:59 IST • Weightage: *30%* of the course grade

In this project you build a QR Code decoder. Your hardware is given the grid of dark and light squares of a QR Code and must produce the message stored in it, correcting any damage along the way.

Section 1 explains how a QR Code stores its message. It contains everything you need for the required part of the project. The rest of the document specifies the interface, what is tested, and how your design is graded.

1 The QR Code

1.1 Symbol structure

A QR Code *symbol* is a square grid of *modules*. A module is one square, either dark (1) or light (0). We address a module by its row r and column c, both counted from 0 at the top-left corner.

The size of the grid is given by the *version* v, a number from 1 to 40. A version-v symbol has $N = 17 + 4v$ modules per side, so version 1 is 21×21 and version 40 is 177×177.

Not every module carries data.

- three **finder patterns**, the 7×7 squares in the top-left, top-right and bottom-left corners, each with a one-module light **separator** on its inner sides;
- two **timing patterns**, row 6 and column 6 between the finder patterns, alternating dark and light;
- the **format information**, 15 bits stored twice (Section 1.2);
- one module at $(N - 8,\ 8)$ that is always dark.

Larger versions add two more kinds of fixed module:

- **alignment patterns**, 5×5 squares, from version 2 up. Their centres lie on a grid of coordinates that depends on the version: version 2 has the single centre $(18, 18)$, version 7 has six.
- **version information**, two 6×3 areas next to the top-right and bottom-left finder patterns, from version 7 up.

We call all of these *function modules*. Every other module is a *data module*. **This resource might be useful to visualize how the patterns are placed.**

1.2 Format information

Before the data can be read, the decoder needs two settings:

- the **error correction level**, 2 bits, which fixes how many of the symbol's bytes are data and how many are check bytes: `01` is L, `00` is M, `11` is Q and `10` is H;
- the **mask number**, 3 bits, which says which of eight patterns was XORed over the data modules.

These five bits $d_4 \ldots d_0$ (level first) are protected by a BCH$(15, 5)$ code. The encoder appends ten zeros to the five bits, divides by the generator `10100110111` in GF(2), and uses the 10-bit remainder as check

bits. The resulting 15 bits are then XORed with the constant `101010000010010`, so that no setting is stored as fifteen light modules.

Only 32 of the 2^{15} possible values can therefore appear, and any two of them differ in at least 7 positions. A decoder that picks the valid value nearest to what it read recovers the settings whenever at most 3 of the 15 modules were misread. If the nearest valid value is more than 3 positions away, the read cannot be trusted.

The 15 bits $f_{14} \ldots f_0$ are stored twice, at the positions of Table 1.

Bit	**First copy** (r, c)	**Second copy** (r, c)
$f_0 \ldots f_5$	$(i,\ 8)$ for $i = 0 \ldots 5$	$(8,\ N-1-i)$
f_6	$(7,\ 8)$	$(8,\ N-7)$
f_7	$(8,\ 8)$	$(8,\ N-8)$
f_8	$(8,\ 7)$	$(N-7,\ 8)$
$f_9 \ldots f_{14}$	$(8,\ 14-i)$ for $i = 9 \ldots 14$	$(N-15+i,\ 8)$

Table 1: Where format bit f_i is stored. The second copy runs along row 8 for $i \le 7$ and down column 8 for $i \ge 8$.

1.3 Masks and reading order

The encoder XORs one of the eight patterns of Table 2 over the data modules, so that the symbol has no large blank areas. If the condition for the chosen mask for the original data module at (r, c) was true initially, it was inverted in the encoding. Your decoder module must invert it again to recover the original form. Function modules are never masked.

Mask	**Inverted where**	**Mask**	**Inverted where**
0	$(r + c) \bmod 2 = 0$	4	$(\lfloor r/2 \rfloor + \lfloor c/3 \rfloor) \bmod 2 = 0$
1	$r \bmod 2 = 0$	5	$(rc \bmod 2) + (rc \bmod 3) = 0$
2	$c \bmod 3 = 0$	6	$((rc \bmod 2) + (rc \bmod 3)) \bmod 2 = 0$
3	$(r + c) \bmod 3 = 0$	7	$(((r + c) \bmod 2) + (rc \bmod 3)) \bmod 2 = 0$

Table 2: The eight mask patterns.

The data modules are read in the zigzag order:

1. Start with the two rightmost columns, $N - 1$ and $N - 2$, at the bottom row, moving upwards.
2. In each row of the strip, take the right-hand module first, then the left-hand one.
3. At the end of a strip, move two columns to the left and reverse direction.
4. Column 6 holds a timing pattern and is not part of any strip: after the strip of columns 8 and 7 comes the strip of columns 5 and 4.
5. Skip every function module. It is not read and does not count.

The unmasked bits, taken eight at a time with the first bit as the most significant, form the symbol's bytes, which the standard calls *codewords*. A version-1 symbol has 208 data modules and so 26 bytes. Some versions have a few data modules left over after the last byte; ignore them.

1.4 Blocks and Reed–Solomon

The bytes of a symbol are divided into one or more *blocks*. Each block is a Reed–Solomon codeword: some data bytes followed by e check bytes. A block with e check bytes can have up to $\lfloor e/2 \rfloor$ of its bytes corrupted, anywhere in the block, and still be recovered.

The block structure depends on the version and the level. Table 3 gives it for versions 1 to 4; the supplied package `QRTables` has it for all 40 versions.

Ver.	Level	Blocks	Data	Check	Ver.	Level	Blocks	Data	Check
1	L	1	19	7	3	L	1	55	15
1	M	1	16	10	3	M	1	44	26
1	Q	1	13	13	3	Q	2	17	18
1	H	1	9	17	3	H	2	13	22
2	L	1	34	10	4	L	1	80	20
2	M	1	28	16	4	M	2	32	18
2	Q	1	22	22	4	Q	2	24	26
2	H	1	16	28	4	H	4	9	16

Table 3: Block structure for versions 1 to 4. *Data* and *Check* are bytes per block.

Interleaving. When a symbol has more than one block, the blocks are not stored one after another. The symbol holds the first data byte of every block, then the second data byte of every block, and so on, followed by the check bytes interleaved in the same way. From version 5 up, a symbol can have blocks of two lengths: n_1 blocks with d data bytes, followed by n_2 blocks with $d + 1$. The extra data byte of each longer block is placed after all the others. With $n = n_1 + n_2$ blocks, byte i of block b is therefore the k-th byte read from the symbol, where

$$k = \begin{cases} i\,n + b & \text{for data byte } i < d, \\ d\,n + (b - n_1) & \text{for data byte } i = d \text{ (longer blocks only)}, \\ D + j\,n + b & \text{for check byte } j, \end{cases}$$

and D is the total number of data bytes in the symbol.

The code. The arithmetic is in GF(2^8) (see next paragraph) with the polynomial $x^8 + x^4 + x^3 + x^2 + 1$ (`0x11D`) and the primitive element $\alpha = 2$. Addition is XOR. Treat a block of m bytes as the polynomial $R(x) = r_{m-1}x^{m-1} + \cdots + r_1 x + r_0$, where r_{m-1} is the first byte of the block. An undamaged block satisfies

$$R(\alpha^j) = 0 \qquad \text{for } j = 0, 1, \ldots, e - 1.$$

The values $S_j = R(\alpha^j)$ are the *syndromes*. If they are all zero the block is clean. Otherwise they depend only on the errors, and from them a decoder finds which bytes are wrong and by how much. The usual route is Berlekamp–Massey for the error locator, a Chien search for its roots, and Forney's formula for the error values, but any correct method is acceptable.

Note: GF(2^8) represents the corresponding Galois-Field, where the addition operation degenerates to a logical XOR modulo the characteristic polynomial. You can learn more about Galois-field arithmetic in this web-page.

1.5 The payload

The corrected data bytes of all blocks, block 0 first, form one bit stream, most significant bit first. The stream is a sequence of *segments*. Each segment has three parts:

1. a 4-bit **mode indicator**;
2. a **character count**, whose width is given in Table 4;
3. the characters, packed as the mode specifies.

The message ends at a terminator, or when fewer than 4 bits of the stream remain. Whatever follows the terminator is padding and must be ignored.

Mode	Indicator	Count width, by version 1–9	10–26	27–40	Packing
Numeric	`0001`	10	12	14	Three digits in 10 bits. A final group of two digits takes 7 bits, of one digit 4 bits.
Alphanumeric	`0010`	9	11	13	Two characters a, b in 11 bits as $45a + b$. A final single character takes 6 bits.
Byte	`0100`	8	16	16	One byte in 8 bits.
Terminator	`0000`		none		End of the message.

Table 4: Segment modes. The alphanumeric character set, in order of value 0 to 44, is `0`–`9`, `A`–`Z`, space, and `$ % * + - . / :`

2 The decoder

Your decoder sits between two parts of our test bench (Figure 1). On one side is a **bitmap memory** that holds the symbol. Your decoder reads it through a memory port that is part of its own interface, one module per clock cycle. On the other side, the test bench collects the decoded bytes.

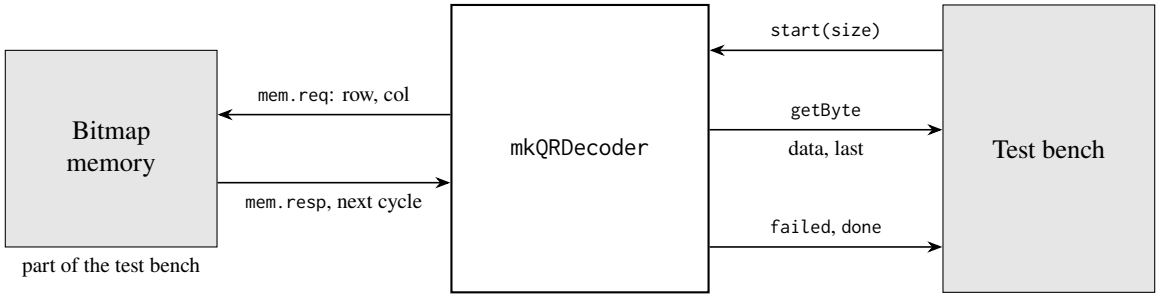


Figure 1: The decoder and its surroundings. Both shaded boxes are ours.

3 Interface skeleton

You are given `QRTypes.bsv`, which defines the output type and the two interfaces of Figure 1, and `QRTables.bsv`, which holds the block structure of every version. Do not modify either file: we compile your code against our own copies.

```
package QRTypes;

typedef UInt#(8) Coord;    // row, column or side length

typedef struct {
   Bit#(8) data;
   Bool    last;           // set on the final byte of the message
} OutByte deriving (Bits, Eq, FShow);

// The decoder's port to the bitmap memory.
interface MemPort;
   method ActionValue#(Tuple2#(Coord, Coord)) req();  // (row, col)
   (* always_ready, always_enabled *)
   method Action resp(Bit#(1) b);                     // 1 = dark
endinterface

interface QRDecoder;
   method Action               start(Coord size);
```

```
   method ActionValue#(OutByte) getByte();
   method Bool                  failed();
   method Bool                  done();
   interface MemPort            mem;
endinterface

endpackage
```

`QRDecoder.bsv` contains the module you fill in:

```
package QRDecoder;

import QRTypes  :: *;
import QRTables :: *;

(* synthesize *)
module mkQRDecoder (QRDecoder);

   // Read the symbol through mem, decode it, and return the message
   // through getByte.

endmodule

endpackage
```

Keep the module name, its interface and its `(* synthesize *)` attribute. Everything behind the interface is yours, including any packages and modules of your own that you submit alongside it.

4 Functional specification

4.1 What the test bench guarantees

A1. The bitmap memory holds the whole symbol before `start` fires, and does not change until the decode ends.

A2. The symbol is upright and not mirrored, with its top-left module at $(0, 0)$ and no quiet zone around it.

A3. `start` fires only when the decoder is idle: after reset, or once `done` or `failed` has risen.

A4. Damage takes the form of flipped modules, anywhere in the symbol, function modules included.

A5. A block with more than $\lfloor e/2 \rfloor$ damaged bytes differs in more than $\lfloor e/2 \rfloor$ bytes from every valid block. A correct decoder therefore detects the damage and never miscorrects.

By A2 and A4, you need not locate the symbol, and you must not reject it because a finder, timing or alignment pattern is damaged. The version follows from the side length, so read only the format information and the data modules.

4.2 What your decoder must do

G1. **Version.** Find v from the side length N given to `start`. If $N \neq 17 + 4v$ for every $1 \leq v \leq 40$, fail.

G2. **Format.** Read both copies of the format information and take the one nearer to a valid value, the first copy on a tie. If it is more than 3 positions away, fail.

G3. **Correction.** Read and unmask the data modules (Section 1.3), de-interleave the blocks and correct each one (Section 1.4). If a block cannot be corrected, fail.

G4. **Payload.** Decode the segments (Section 1.5) in order and return one byte per character: its ASCII code in numeric and alphanumeric mode, the byte itself in byte mode. Fail on a malformed payload: an unknown mode indicator, a segment that runs past the end of the data, or a numeric or

alphanumeric group whose value is out of range, such as three digits above 999.

To fail is to raise `failed`. The test bench then discards any bytes already returned for that symbol.

4.3 Interface and timing contract

Contract

C1. `getByte` may fire only when a decoded byte is genuinely available. Bytes come out in message order, each exactly once, and only the final one has `last` set.

C2. `failed` and `done` are both False after reset, and from the cycle after `start` fires until the decode ends. Then exactly one of them rises and stays high until the next `start`. `done` may rise only once the byte with `last` set has been taken, or at once for an empty message. `failed` may rise at any point, even after bytes have been returned.

C3. Once `failed` has risen, `getByte` does not fire again until the next `start`.

C4. `mem.req` is ready only in a cycle in which the decoder wants a module, and the test bench always takes the request in that cycle. The answer arrives through `mem.resp` in the next cycle and holds until the answer to the next request replaces it. Modules may be read in any order and as often as you like, at most one per cycle.

C5. `start` and `getByte` express their conditions as *implicit* conditions. Do not attach `always_ready` or `always_enabled` to either.

The test bench applies back-pressure: it sometimes leaves `getByte` uncalled for many cycles, and your decoder must hold its output until it is taken. A decode that has not ended within a generous cycle limit fails that test case. To measure latency, the test bench takes each byte as soon as it is ready and counts the cycles from the one in which `start` fires to the one in which the byte with `last` set is taken.

5 Verification

A decoder that works on the first few QR codes you try, can still be wrong in dozens of ways, and our test suite will find them. Verification is how you find them first, and it carries marks of its own (Table 7). You have complete freedom in how you go about it. What follows is one route that works well.

5.1 Begin with a software decoder

Before you write any BSV, write the whole decoder in Python, or any language you like. It is the quickest way to really understand Section 1, and it becomes your *reference model*: the thing every piece of RTL is compared against. Libraries such as `segno` or `qrcode` will encode a message of your choice and hand you the module matrix, so you can try your model on thousands of real symbols before you trust it.

Have the model print its intermediate results as well: the format bits, the raw codewords, the syndromes, each block before and after correction. When the RTL disagrees with the model later, these tell you *where* the two first part ways, not just that they do.

5.2 Test each unit on its own

Split the design into pieces that each do one job, and test each piece against the matching function of your model before you connect anything. A bug is far easier to find in a fifty-line module than in the full decoder. One natural split is:

- GF(2^8) arithmetic;
- the format decoder: 15 bits in, level and mask (or failure) out;

- the reader: the order in which modules are visited, and the unmasking;
- the de-interleaver;
- the Reed–Solomon stages: syndromes, error locator, error positions and values;
- the payload parser: bytes in, characters out.

Some of these have so few inputs that you can test *every* one. A GF(2^8) multiplier has 2^{16} input pairs and the format decoder 2^{15}: test them all and you are done with those units for good. For the rest, combine random tests with the hand-picked cases you think are hardest.

You will be working in teams of two. Split the work so that the person who writes the model of a unit never also designs its RTL. If one person does both, a misreading of the spec goes into both, and the comparison passes anyway. The same holds for AI tools: a model and an RTL module generated from the same prompt will share the same mistakes.

5.3 Then test the whole decoder

Build a test bench that plays the part of ours: it holds a bitmap, answers `mem.req` one cycle later, collects the bytes and checks them against your model. Make it unfriendly. Stall `getByte` for random stretches, decode several symbols in a row without a reset, and check C1–C5 as you go. Your test set should include at least:

- every version, level and mask, though not every combination;
- every mode, messages with several segments, an empty message, and the short final groups (one or two digits, one alphanumeric character);
- exactly $\lfloor e/2 \rfloor$ damaged bytes in every block, damage in check bytes as well as data bytes, and damaged function modules;
- up to 3 wrong bits in one or both copies of the format information;
- symbols that must fail: one damaged byte too many in a block, format information past repair, a bad side length, a malformed payload.

There are two common ways to connect the model and the RTL. The model can write test vectors and expected outputs to files, which a BSV test bench reads (for instance with `mkRegFileLoad`) and checks in Bluesim. Or you can drive the Verilog that `bsc` generates straight from Python, using `cocotb` on Verilator. The course Docker image has both.

5.4 Know what you have covered*

Passing tests only tell you about the cases you thought of. Coverage tells you about the rest. Verilator's `--coverage` option reports which parts of the generated Verilog your tests exercised, and `cocotb-coverage` lets you count the cases you care about, such as each version, level, mask and mode, or each number of errors in a block. Anything nobody reached is either a test you still need to write or logic you do not need.

Keep everything runnable with one command, such as `make test`, and turn every bug you find into a test so that it stays fixed. Put it all in `verification/`, with a README that says how to run it. In your report, say what you tested, how many tests ran, what coverage you reached and which bugs the tests caught.

***Only for you to design better , not a part of grading**

6 Evaluation

6.1 RTL

Evaluation for this assignment would largely be based on correctness and the majority of the score alloted in proportion to the fraction of testcases passed. Your submitted files should align with the provided skeleton. Your module will be dropped inside a test suite that instantiates your module and tests with multiple input QR patterns. Ensure that you cover all cases, including but not limited to:

1. Valid QR codes of all versions.
2. Full sized QR codes with simulated errors (this means some of the modules in the QR code may be plain white or black).
3. QR codes with errors beyond the correctable amount - you should raise the error flag as specified above.

All testcases will be graded in a binary manner - your output file will be diffed against the reference output for each of the testcase, and that testcase is considered correct only if there are no differences.

Additionally, you are expected to thoroughly verify your design and take it through the physical design flow, as outlined below. A broad split of points for the project can be also be found below.

6.2 Physical Design

6.2.1 Power Performance and Area

In a design, PPA represents Power, Performance, and Area. These are three key metrics used to evaluate the quality and efficiency of a design.

The architecture and process technology influences the PPA of the target design. Of course, you as designers want the least power and provide the fastest performance while occupying the smallest amount of area, but in practical engineering trade-offs are unavoidable. It is very often not possible to achieve the lowest power, highest performance, and smallest area simultaneously. **Hence, you as a designer must come to an optimal PPA number considering the tradeoffs.**

6.2.2 Iteration for achieving optimal PPA

You are expected to take every design through a full round of iteration (Design –> Synthesis –> Analyze –> Re-Design). The Goal is to improve the Power, Performance and Area (PPA) numbers of your design after analyzing the synthesis reports.

1. Make sure you run synthesis for each round with 40 percentage clock period for external delay.
2. Feel free to re-use the udcounter example scripts. Do not change the timing libraries used in the example.
3. Synthesis to be run using Design Compiler and it should be normal synthesis, not the topographical synthesis
4. After each iteration fill in the table 5 below.

6.2.3 Place and Route

For the design which you think is the best optimal PPA number, you are expected to take it through the Place and Route Flow.

1. Feel free to re-use the udcounter example scripts. Do not change the timing libraries used in the example.
2. PNR should be done using ICC tool and not ICC2 or Fusion Compiler
3. Layout should be DRC and LVS clean.
4. After completion of Place and Route fill the below table - 6.

A well-optimized design after Place and Route is characterized by minimal PPA degradation relative to the best synthesis result, while maintaining the timing, power, and area constraints.

6.2.4 Tables to Be filled

Table values to be submitted in a seperate pd_report.pdf

Design Number	Critical Path Clk Period*	Critical Path Slack*	Max Clock Freq**	Total Number of Leaf Cells	Total number of sequential cells	Total power	Design Area
1.							
2.							
3.							
4.							
5.							

Table 5: Post - Synthesis QoR Data

Design Number	Critical Path Clk Period*	Critical Path Slack*	Max Clock Freq**	Total Number of Leaf Cells	Total number of sequential cells	Total power	Block Dimensions of the Layout	Design Area
1.								

Table 6: Post - Routing QoR Data

* Fill Critical Path Clk Period, Critical Path Slack from f2f path
** Clock period specified in the SDC, without negative slack
"report_power" command should be used for obtaining power related metrics

A submission that fails to compile scores zero, so leave time to check that the files you upload are the files you tested.

6.3 Submission format

```
<Roll_Number>/
|-- README.md
|-- report.pdf
|-- pd_report.pdf
|-- bsv/
|   |-- QRDecoder.bsv           your design
|   |-- ...                     any other .bsv files of your own
|   |-- QRTypes.bsv             supplied, replaced by ours
|   |-- QRTables.bsv            supplied, replaced by ours
|-- verification/
`-- pd/
    |-- design_1/
          |-- design_1_dc_qor.rpt
          |-- design_1_dc_timing.rpt
          |-- design_1_dc_power.rpt
          |-- design_1_dc.tcl
          |-- design_1_dc.sdc
          |-- dc_run.log
    |...(4 designs)
    |-- best_design/
          |-- best_design_dc_qor.rpt
          |-- best_design_dc_timing.rpt
          |-- best_design_dc_power.rpt
          |-- best_design_dc.tcl
          |-- best_design_dc.sdc
          |-- best_dc_run.log
          |-- best_design_post_route_qor.rpt
          |-- best_design_post_route_power.rpt
          |-- best_design_route.tcl
          |-- best_design_route.sdc
          |-- best_icc_run.log

Please submit the reports and files in the same format and structure.
```

6.4 Mark Split

Component	Marks
Functional test cases, in proportion to the fraction passed	10
Verification (test set size, unit-tests)	5
Design space exploration: at least four iterations, post-synthesis with quantifiable improvement	10
Place and Route: for the best design	5
Total	**30**

Table 7: Mark distribution.

7 Use of AI tools

You may use AI tools as much as you like. We encourage educated and judicious use of those tools, under the condition that you own everything you submit. Owning something in this context means that you

understand every line of your design and testbenches and be able to explain why it exists. You should also be able to explain your microarchitecture, and any decisions you made in the design. All numbers submitted as part of your reports should be real results and not fabricated.

Remember that the goal of this project, especially today, is to teach you to design and think, while also to work effectively with AI. It also provides you with an opportunity to extensively work with physical design tools. **You should be proud of your work if at the end you feel that: you understand how QR codes work, how data is encoded and decoded in QR codes for different versions and error correction levels, how Reed-Solomon code works, Galois-field arithmetic and their significance in data encoding. Additionally, this project should serve as a strong test of your ability to use physical design tools for synthesis and placement and routing, while letting you develop intuition for what kinds of improvements you can expect as you perform design space exploration.**

TL;DR: Don't let your AI agent have all the fun!

8 Academic honesty

Discussing the specification and the language with your classmates is fine; sharing code, in either direction, is not, whether a person or an AI tool wrote it. Submissions are compared against each other and against public repositories, and a match is referred to the institute's disciplinary process without exception.

9 Contact

Questions about the specification should go to the course forum so that everyone sees the answer; anything that would reveal part of your solution should come by email instead.

Sanjeev Subrahmaniyan ee23b102@smail.iitm.ac.in
Abhinav I S ee23b002@smail.iitm.ac.in

For doubts regarding Physical Design, Docker Setup and Server Access feel free to email the following TAs

S K Hari Kheshav cs24s025@cse.iitm.ac.in
Jayanth R ns25z033@smail.iitm.ac.in